

Краткий справочник по некоторым функциям языка C++11

В новый стандарт C++ принесена не только поддержка конкурентности; в нем также есть множество других функций языка и новых библиотек. В этом приложении дадим краткий обзор новых функций языка, используемых в Thread Library и в остальной части книги. Кроме модификатора `thread_local` (рассматриваемого в разделе A.8), ни одна из них не имеет непосредственной связи с конкурентностью. Тем не менее они важны и (или) полезны для создания многопоточного кода. Их перечень я ограничил теми функциями, которые либо необходимы (как, например, ссылки на *r*-значения), либо упрощают код или делают его понятнее. Поначалу возможны затруднения с пониманием кода, использующего эти функции, из-за его новизны, но по мере ознакомления с ним можно будет согласиться с тем, что в целом эти функции делают код понятнее. По мере роста востребованности C++11 код, использующий эти функции, будет встречаться все чаще.

А теперь без лишних предисловий приступим к рассмотрению ссылок на *r*-значения, которые широко применяются Thread Library для облегчения передачи прав владения (потоками, блокировками или чем-либо еще) между объектами.

A.1. Ссылки на *r*-значения

Каждый, кто когда-либо сталкивался с программированием на C++, знаком со ссылками. Ссылки C++ позволяют создавать новые имена для существующих объектов. Все обращения или изменения, совершаемые по новой ссылке, воздействуют на исходный объект, например:

```
int var=42;
int& ref=var;
ref=99;
assert(var==99);
```

← Создание ссылки на `var`

← Исходное значение обновлено в результате присваивания нового значения по ссылке

Единственными ссылками до появления C++11 были *ссылки на l-значения*. Понятие *l*-значения пришло из языка C и относится к тому, что может быть слева от знака присваивания, к именованным объектам, объектам, размещаемым в стеке или в куче, или к компонентам других объектов, то есть ко всему, имеющему вполне определенное место хранения. Понятие *r*-значения также пришло из языка C и относится к тому, что может появляться только справа от знака присваивания — к примеру, к литералам и временным значениям. Ссылки на *l*-значения могут быть привязаны только к *l*-, но не к *r*-значениям. К примеру, нельзя написать

```
int& i=42;    ← Не пройдет компиляцию
```

потому что 42 является *r*-значением. Это верно лишь отчасти, поскольку всегда можно привязать *r*-значение к константной ссылке на *l*-значение:

```
int const& i=42;
```

По это исключение в часть стандарта было введено преднамеренно до появления ссылок на *r*-значения, чтобы дать возможность передавать временные значения функциям, принимающим ссылки. Тем самым разрешаются подразумеваемые преобразования, позволяющие написать следующий код:

```
void print(std::string const& s);
print("hello");
```

← Создание временного
объекта std::string

В стандарте C++11 появились *ссылки на r-значения*, которые привязываются *только* к *r*-, но не к *l*-значениям и объявляются с использованием не одного, а двух амперсандов:

```
int&& i=42;
int j=42;
int&& k=j;    ← Не пройдет компиляцию
```

Чтобы определить, являются ли параметры функции *l*- или *r*-значениями, можно воспользоваться переопределением функции, и одно ее определение будет принимать ссылку на *l*-значение, а другое — ссылку на *r*-значение. Это станет красугольным камнем *семантики перемещений*.

А.1.1. Семантика перемещений

r-значения обычно представлены временными объектами, поэтому в них свободно можно вносить изменения. Если известно, что параметром вашей функции является *r*-значение, им можно воспользоваться в качестве временного хранилища или «украсть» его содержимое без ущерба для корректности программы. Это означает, что вместо *копирования* содержимого параметра, являющегося *r*-значением, можно *переместить* его содержимое. Для крупных динамических структур это позволит сэкономить на многократном распределении памяти и предоставит широкие возможности для их оптимизации. Рассмотрим функцию, получающую в качестве параметра `std::vector<int>` и нуждающуюся во внутренней копии для изменения,

не затрагивая при этом оригинал. Раньше для этого нужно было принять параметр как `const`-ссылку на `l`-значение и сделать внутреннюю копию:

```
void process_copy(std::vector<int> const& vec_)
{
    std::vector<int> vec(vec_);
    vec.push_back(42);
}
```

Это позволяет функции принимать как `l`-, так и `r`-значения, но в любом случае вынуждает выполнять копирование. Если переопределить функцию с версией, принимающей ссылку на `r`-значение, то в случае с `r`-значением можно избежать копирования, поскольку известно, что в оригинал разрешается свободно вносить изменения:

```
void process_copy(std::vector<int> && vec)
{
    vec.push_back(42);
}
```

Теперь, если рассматриваемая функция является конструктором вашего класса, можно будет «украсть» внутреннее содержимое `r`-значения и воспользоваться им для ваших новых экземпляров. Рассмотрим класс, показанный в листинге А.1. В конструкторе по умолчанию он распределяет весьма крупный блок памяти, высвобождаемый в деструкторе.

Листинг А.1. Класс с конструктором перемещения

```
class X
{
private:
    int* data;
public:
    X():
        data(new int[1000000])
    {}
    ~X()
    {
        delete [] data;
    }
    X(const X& other): ← ❶
        data(new int[1000000])
    {
        std::copy(other.data, other.data+1000000, data);
    }
    X(X&& other): ← ❷
        data(other.data)
    {
        other.data=nullptr;
    }
};
```

Копирующий конструктор ❶ определяется вполне ожидаемо: распределение памяти под новый блок и копирование в него данных. Но есть также и новый кон-

структур, получающий прежнее значение по ссылке на *r*-значение ❷. Это *перемещающий конструктор*. В этом случае копируется *указатель* на данные, а в экземпляре объекта `other` остается нулевой указатель, благодаря чему экономится крупный блок памяти и время на создание переменных из *r*-значений.

Для класса `X` перемещающий конструктор является оптимизацией, но в некоторых случаях есть смысл предоставить такой конструктор, даже когда совершенно бессмысленно предоставлять копирующий конструктор. Например, вся суть `std::unique_ptr<>` состоит в том, что каждый ненулевой экземпляр является единственным указателем на свой объект, поэтому в копирующем конструкторе нет никакого смысла. А вот перемещающий конструктор позволяет передавать владение указателем между экземплярами и использовать `std::unique_ptr<>` в качестве возвращаемого значения функции — указатель *не копируется, а перемещается*.

Если требуется явное перемещение из поименованного объекта, о котором известно, что больше он не будет использоваться, его можно привести к типу *r*-значения либо воспользовавшись `static_cast<X&&>(x2)`, либо вызвав `std::move()`:

```
X x1;
X x2=std::move(x1);
X x3=static_cast<X&&>(x2);
```

Это может пригодиться, когда нужно переместить значение параметра в локальную или компонентную переменную без копирования, поскольку, несмотря на то что параметр, представляющий собой ссылку на *r*-значение, может быть привязан к *r*-значениям, внутри функции он рассматривается как *l*-значение:

```
void do_stuff(X&& x_)
{
    X a(x_);           ← Копируется
    X b(std::move(x_)); ← Перемещается
}
do_stuff(X());        ← Все верно; r-значение привязывается
X x;                  ← к ссылке на r-значение
do_stuff(x);          ← Ошибка; l-значение не может
                        ← привязываться к ссылке на r-значение
```

Семантика перемещения широко используется в Thread Library, как в тех случаях, когда копирование не имеет смысла. По ресурсы можно передавать и в качестве оптимизации для исключения весьма затратного копирования, когда источник все равно подлежит уничтожению. Пример я уже приводил в разделе 2.2, где функция `std::move()` использовалась для передачи экземпляра `std::unique_ptr<>` в только что созданный поток, а также в разделе 2.3, где показана передача владения потоками между экземплярами `std::thread`.

Экземпляры классов `std::thread`, `std::unique_lock<>`, `std::future<>`, `std::promise<>` и `std::packaged_task<>` нельзя скопировать, но во всех их классах есть перемещающие конструкторы, позволяющие передавать связанный с ними ресурс между экземплярами и поддерживающие их использование в качестве значений, возвращаемых функцией. Экземпляры классов `std::string` и `std::vector<>` всегда можно скопировать, но у них также есть перемещающие конструкторы и связанные

с перемещениями операторы, позволяющие избегать копирования больших объемов данных из *r*-значения.

Стандартная библиотека C++ никогда ничего не делает с объектом, явно перемещенным в другой объект, кроме его уничтожения или присваивания ему значения (либо путем копирования, либо, что более вероятно, путем перемещения). Но правилом хорошего тона будет гарантировать в инварианте класса наличие состояния «перемещен из». Например, экземпляр `std::thread`, использованный в качестве источника для перемещения, эквивалентен созданному по умолчанию экземпляру `std::thread`, а экземпляр `std::string`, использованный в качестве источника для перемещения, по-прежнему будет иметь допустимое состояние, хотя не дается никаких гарантий насчет того, каким именно будет это состояние (в понятиях длины строки или содержащихся в ней символов).

A.1.2. Ссылки на *r*-значения и шаблоны функций

Существует еще один, последний, нюанс, имеющий отношение к использованию ссылок на *r*-значения в качестве параметров шаблона функции: если параметр функции является ссылкой на *r*-значение, представляющее собой параметр шаблона, то механизм автоматического вывода типа аргумента шаблона делает вывод, что типом будет ссылка на *l*-значение, если предоставлено *l*-значение, или простой обычный тип, если предоставлено *r*-значение. Последнюю фразу не просто понять, поэтому обратимся к примеру. Рассмотрим следующую функцию:

```
template<typename T>
void foo(T&& t)
{}
```

Если ее вызвать, как показано далее, с *r*-значением, то будет сделан вывод, что *T* является типом значения:

```
foo(42);           ← Вызывает foo<int>(42)
foo(3.14159);     ← Вызывает foo<double>(3.14159)
foo(std::string()); ← Вызывает foo<std::string>(std::string())
```

Но если вызвать `foo` с *l*-значением, то будет сделан вывод, что *T* является ссылкой на *l*-значение:

```
int i=42;
foo(i); ← Вызывает foo<int&>(i)
```

Поскольку параметр функции объявлен как `T&&`, получается, что это ссылка на ссылку, что рассматривается как исходный ссылочный тип. Сигнатура `foo<int&>()` такова:

```
void foo<int&>(int& t);
```

Это позволяет одному шаблону функции принимать в качестве параметров как *l*-, так и *r*-значение, что используется конструктором `std::thread` (см. разделы 2.1 и 2.2), и тогда, если параметр является *r*-значением, предоставляемый вызываемый объект можно не скопировать, а переместить.

А.2. Удаленные функции

Иногда разрешение копировать класс не имеет никакого смысла. Ярким примером этого может послужить `std::mutex`. Что бы получилось, если скопировать мьютекс? Еще один пример: `std::unique_lock<>` — экземпляр является единственным владельцем удерживаемой им блокировки. Если по-настоящему его скопировать, это будет означать, что копия также удерживает блокировку, что не имеет никакого смысла. Передача владения, описанная в подразделе А.1.2, имеет смысл, но это не копирование. Уверен, что вы можете привести и другие примеры.

Стандартной манерой предотвращения копирования класса является объявление копирующего конструктора и оператора копирующего присваивания закрытыми с последующим непредоставлением их реализации. Тогда, если код вне рассматриваемого класса попытается скопировать экземпляр, будет выдана ошибка компиляции, а если это же попытается сделать какая-нибудь компонентная функция класса, будет выдана ошибка на этапе компоновки (из-за отсутствия реализации):

```
class no_copies
{
public:
    no_copies(){}
private:
    no_copies(no_copies const&);
    no_copies& operator=(no_copies const&);
};
no_copies a;
no_copies b(a);
```

Нет реализации

← Не пройдет компиляцию

В комитете по разработке стандарта C++11 понимали, что это широко распространенный прием, и осознавали его невысокую надежность. Поэтому был предоставлен более общий механизм, применимый также и в других классах: функцию можно указать удаленной, добавив к ее объявлению признак `= delete`. Это позволяет перенести `no_copies` следующим образом:

```
class no_copies
{
public:
    no_copies(){}
    no_copies(no_copies const&) = delete;
    no_copies& operator=(no_copies const&) = delete;
};
```

Это намного нагляднее по сравнению с исходным кодом и четче выражает намерение, а также позволяет компилятору выдавать более внятные описания ошибок и перенести момент выдачи ошибки из этапа компоновки в этап компиляции в том случае, если вам вздумается выполнить копирование внутри компонентной функции вашего класса.

Если в дополнение к удалению копирующего конструктора и оператора копирующего присваивания будет также явно написан перемещающий конструктор и оператор перемещающего присваивания, ваш класс будет допускать только перемещение по аналогии с классами `std::thread` и `std::unique_lock<>`. Пример такого типа, предназначенного только для перемещений, показан в листинге А.2.

Листинг А.2. Простой тип, предназначенный только для перемещения

```

class move_only
{
    std::unique_ptr<my_class> data;
public:
    move_only(const move_only&) = delete;
    move_only(move_only&& other):
        data(std::move(other.data))
    {}
    move_only& operator=(const move_only&) = delete;
    move_only& operator=(move_only&& other)
    {
        data=std::move(other.data);
        return *this;
    }
};
move_only m1;
move_only m2(m1);
move_only m3(std::move(m1));

```

← Ошибка; копирующий конструктор объявлен удаленным

← Все в порядке; перемещающий конструктор найден

Объекты, допускающие только перемещения, можно передавать в качестве параметров функциям и возвращать из функций. Но если потребуется переместить их из *l*-значения, это намерение нужно явно выразить, воспользовавшись функцией `std::move()` или оператором `static_cast<T&&>`.

Спецификатор `= delete` можно применять к любой функции, не только к копирующим конструкторам и операторам присваивания. Тем самым поясняется, что функция недоступна. Но это еще не все. Удаленная функция, как и любая другая, участвует в разрешении переопределения, и при ее выборе возникает ошибка компиляции. Этим можно воспользоваться для удаления конкретных переопределений. Например, если ваша функция принимает параметр типа `short`, то сужение значений типа `int` можно предотвратить, написав переопределение, принимающее `int`, и объявив его удаленным:

```

void foo(short);
void foo(int) = delete;

```

Теперь любая попытка вызова `foo` с типом `int` будет встречена выдачей ошибки компиляции, и вызывающему коду придется выполнить явное преобразование предоставленного значения в `short`:

```

foo(42);
foo((short)42);

```

← Ошибка; переопределение для типа `int` объявлено удаленным

← Все в порядке

А.3. Функции по умолчанию

Удаленные функции позволяют явно указать на то, что функция не реализована, а у *функций по умолчанию* (defaulted) обратное предназначение: указать на то, что компилятор должен написать функцию вместо вас, применив реализацию по умол-

чанию (исходную). Сделать это можно только в отношении тех функций, которые компилятор может автоматически создавать в любом случае: конструкторов по умолчанию, деструкторов, копирующих конструкторов, переименовывающих конструкторов, операторов копирующего присваивания и операторов переименовывающего присваивания.

Что может побудить к таким действиям? На это есть несколько причин:

- ❑ *желание изменить доступность функции* — изначально функции, генерируемые компилятором, являются открытыми. Если нужно сделать их защищенными или даже закрытыми, создавать их придется самостоятельно. Если объявить их функциями по умолчанию, можно заставить компилятор написать функцию и изменить уровень ее доступности;
- ❑ *желание воспользоваться функцией в качестве документации* — если сгенерированной компилятором версии вполне достаточно, то, может быть, стоит объявить о ее достаточности явным образом, чтобы вы или кто-то другой, изучая код, понимали, что это сделано намеренно;
- ❑ *желание заставить компилятор сгенерировать функцию, если он не стал бы это делать*, — обычно это делается в отношении конструкторов по умолчанию, которые при иных обстоятельствах создаются компилятором только при отсутствии конструкторов, определенных пользователем. Если нужно, к примеру, определить специализированный копирующий конструктор, можно получить конструктор, сгенерированный компилятором, объявив его исходным (по умолчанию);
- ❑ *желание сделать деструктор виртуальным и при этом оставить его в числе генерируемых компилятором*;
- ❑ *желание принудительно создать конкретное объявление копирующего конструктора, например, чтобы он принимал параметр по умолчанию не по const-ссылке, а по не-const-ссылке*;
- ❑ *желание воспользоваться особыми свойствами сгенерированных компилятором функций, утрачиваемых при предоставлении своей реализации*. Подробное разъяснение вскоре последует.

Если удаленные функции объявляются продолжением объявления спецификатором `= delete`, то функции по умолчанию объявляются продолжением объявления спецификатором `= default`, например:

```
class Y
{
private:
    Y() = default;
public:
    Y(Y&) = default;
    T& operator=(const Y&) = default;
protected:
    virtual ~Y() = default;
};
```

Изменение уровня доступности

Прием не-const-ссылки

Объявление в качестве функции по умолчанию с целью документирования

Изменение уровня доступности и добавление виртуальности

Я уже упоминал, что функции, сгенерированные компилятором, могут иметь особые свойства, которые невозможно было бы получить от версии, определенной пользователем. Наиболее существенное отличие состоит в том, что функция, сгенерированная компилятором, может быть *тривиальной*. Это влечет за собой ряд последствий, включая следующие:

- ❑ объекты с тривиальными копирующими конструкторами, тривиальными операторами копирующего присваивания и тривиальными деструкторами можно копировать с помощью `memcpy` или `memmove`;
- ❑ у литеральных типов, используемых для `constexpr`-функций (см. раздел А.4), должен быть тривиальный конструктор, копирующий конструктор и деструктор;
- ❑ классы с тривиальным конструктором по умолчанию, копирующим конструктором, оператором копирующего присваивания и деструктором можно использовать в связке с конструктором и деструктором, которые определены пользователем;
- ❑ классы с тривиальным оператором копирующего присваивания можно использовать с шаблоном класса `std::atomic<>` (см. подраздел 5.2.6) с целью предоставления значения этого типа с атомарными операциями.

Но просто объявление функции с использованием спецификатора `= default` не делает ее тривиальной — она станет таковой только в том случае, если класс также поддерживает тривиальность всех остальных критериев для соответствующей функции — но явное написание функции в пользовательском коде *не даст ей возможности* стать тривиальной.

Второе отличие классов с функциями, сгенерированными компилятором, от их предоставленных пользователем эквивалентов заключается в том, что класс без конструкторов, предоставленных пользователем, может быть *агрегатным*, а значит, его можно инициализировать с помощью агрегатного инициализатора:

```
struct aggregate
{
    aggregate() = default;
    aggregate(aggregate const&) = default;
    int a;
    double b;
};
aggregate x={42,3.141};
```

В данном случае `x.a` инициализируется значением 42, а `x.b` инициализируется значением 3.141.

Третье отличие функций, сгенерированных компилятором, от их предоставленных пользователем эквивалентов известно немногим и применимо только к конструктору по умолчанию и к конструктору классов по умолчанию, отвечающих конкретному критерию. Рассмотрим следующий класс:

```
struct X
{
    int a;
};
```


Если создается экземпляр класса `X` без инициализатора, содержащееся в нем значение (`a`), относящееся к типу `int`, *инициализируется по умолчанию*. Если у объекта статическая продолжительность хранения, он инициализируется нулевым значением. В противном случае у него получится неопределенное значение, которое потенциально может вызвать неопределенное поведение, если к нему будет обращение, предшествующее присваиванию ему нового значения:

`X x1;` ← `У x1.a имеется неопределенное значение`

Если же инициализировать ваш экземпляр `X` путем явного вызова конструктора по умолчанию, то он будет инициализирован нулевым значением:

`X x2=X();` ← `x2.a==0`

Эта весьма странная возможность также распространяется на базовые классы и компоненты. Если у вашего класса есть конструктор по умолчанию, сгенерированный компилятором, и любые ваши компонентные данные и базовые классы также имеют конструкторы по умолчанию, сгенерированные компилятором, то компонентные данные таких базовых классов и компонентов, являющихся встроенными типами, также либо остаются с неопределенным значением, либо инициализируются нулевым значением, в зависимости от того, есть ли у внешнего класса свой явно вызываемый конструктор по умолчанию.

При всей замысловатости и потенциальной предрасположенности к допущению ошибок это правило находит свое применение. Если вы самостоятельно создали конструктор по умолчанию, эта возможность пропадает; либо компонентные данные, подобные `a`, всегда инициализируются (поскольку вы указали значение `a` или явно вызвали конструктор по умолчанию) или вообще не инициализируются (поскольку вы этого не сделали):

`X::X():a(){}` ← `Всегда a==0`
`X::X():a(42){}` ← `Всегда a==42`
`X::X(){} ❶`

Если вообще опустить инициализацию `a` из конструктора `X`, как в третьем примере ❶, то переменная `a` останется для нестатических экземпляров `X` не проинициализированной, а для экземпляров `X` со статической продолжительностью хранения будет проинициализирована нулевым значением.

При обычных обстоятельствах, если любой другой конструктор будет написан вами самостоятельно, компилятор больше не будет генерировать для вас конструктор по умолчанию, поэтому, если конструктор вам все же нужен, его придется создавать самостоятельно. Следовательно, эта возможность будет утрачена. Но путем явного объявления конструктора по умолчанию можно заставить компилятор генерировать для вас конструктор по умолчанию, и такая возможность сохранится:

`X::X() = default;` ← `Применяемые по умолчанию правила инициализации`

Это актуально для атомарных типов (см. раздел 5.2), у которых есть конструкторы, явно объявленные конструкторами по умолчанию. У них всегда неопределенное значение по умолчанию, если только: а) у них нет статической продолжительности хранения (в силу чего они статически инициализируются нулевым значением); б) вы не выполнили явный вызов конструктора по умолчанию для запроса нулевой инициализации; в) вы не указали значение явным образом. Замечу, что в случае применения атомарных типов, чтобы разрешить статическую инициализацию, конструктор для инициализации со значением объявлен как `constexpr` (см. раздел А.4).

А.4. constexpr-функции

Целочисленные литералы, например 42, являются *константными выражениями*, как и простые арифметические выражения вроде $23 \cdot 2 - 4$. Как частью нового константного выражения можно даже воспользоваться `const`-переменными целочисленного типа, которые сами были инициализированы константными выражениями:

```
const int i=23;
const int two_i=i*2;
const int four=4;
const int forty_two=two_i-four;
```

Для создания переменных, которыми можно воспользоваться в других константных выражениях, кроме использования константных выражений, есть несколько действий, которые могут выполняться *только* с константными выражениями.

- Задание границ массива:

```
int bounds=99;
int array[bounds];
const int bounds2=99;
int array2[bounds2];
```

← Это ошибка, bounds не является константным выражением

← Все верно, bounds2 является константным выражением

- Задание значения параметра шаблона, не являющегося типом:

```
template<unsigned size>
struct test
{};
test<bounds> ia;
test<bounds2> ia2;
```

← Это ошибка, bounds не является константным выражением

← Все верно, bounds2 является константным выражением

- Предоставление инициализатора для компонентных данных класса `static const` целочисленного типа в определении класса:

```
class X
{
    static const int the_answer=forty_two;
};
```

- Предоставление инициализатора для встроеного типа или агрегата, применяемого для статической инициализации:

```

struct my_aggregate
{
    int a;
    int b;
};
static my_aggregate ma1={forty_two,123};
int dummy=257;
static my_aggregate ma2={dummy,dummy};

```

Статическая инициализация

Динамическая инициализация

- Подобная статическая инициализация может применяться во избежание проблем, связанных с порядком проведения инициализации и состояниями гонки.

Здесь нет ничего нового — все это можно делать со стандартом C++ 1998 года издания. Но в той части стандарта C++11, где определяются *константные выражения*, появилось расширение, в котором представлено ключевое слово `constexpr`. В стандартах C++14 и C++17 возможности `constexpr` стали еще шире; но полное описание выходит за рамки данного приложения.

В первую очередь, ключевое слово `constexpr` является модификатором функции. Если параметр и возвращаемое значение функции отвечают определенным требованиям, а тело функции имеет достаточно простую конструкцию, такую функцию можно объявить с модификатором `constexpr`. В таком случае ее допускается применять в константных выражениях, например:

```

constexpr int square(int x)
{
    return x*x;
}
int array[square(5)];

```

В данном случае в массиве `array` будет 25 элементов, поскольку функция `square` объявлена с модификатором `constexpr`. Но лишь то, что функция *может* применяться в константном выражении, еще не означает, что все в ней используемое автоматически является константными выражениями:

```

int dummy=4;
int array[square(dummy)];

```

❶ Ошибка, `dummy` не является константным выражением

В данном примере `dummy` не является константным выражением ❶, следовательно, `square(dummy)` также им не является — это обычный вызов функции, и поэтому он не может использоваться для задания границ массива `array`.

A.4.1. constexpr и типы, определенные пользователем

До сих пор во всех примерах применялись встроенные типы, например `int`. Но новый стандарт C++ позволяет константным выражениям принадлежать к любому типу, удовлетворяющему требованиям, предъявляемым к литеральному типу. Чтобы тип класса мог считаться *литеральным*, необходимо соблюдение следующих условий:

- в классе должен быть тривиальный копирующий конструктор;
- в нем должен быть тривиальный деструктор;

- ❑ все нестатические компонентные данные и базовые классы должны относиться к тривиальным типам;
- ❑ в классе должен быть либо стандартный конструктор по умолчанию, либо конструктор с модификатором `constexpr`, отличный от копирующего конструктора.

Конструкторы с модификатором `constexpr` будут рассмотрены чуть позже. А пока мы сконцентрируемся на классах с тривиальным конструктором по умолчанию, например на классе `CX` из листинга А.3.

Листинг А.3. Класс с тривиальным конструктором по умолчанию

```
class CX
{
private:
    int a;
    int b;
public:
    CX() = default;           ← ❶
    CX(int a_, int b_):      ← ❷
        a(a_), b(b_)
    {}
    int get_a() const
    {
        return a;
    }
    int get_b() const
    {
        return b;
    }
    int foo() const
    {
        return a+b;
    }
};
```

Заметьте, что здесь конструктор по умолчанию явно объявляется *исходным* ❶ с применением спецификатора `= default` (см. раздел А.3), чтобы сохранить его тривиальность, несмотря на наличие конструктора, определенного пользователем ❷. Поэтому данный тип удовлетворяет всем требованиям, позволяющим квалифицировать его в качестве литерального типа, и его можно применить в константных выражениях. Можно, к примеру, предоставить `constexpr`-функцию, создающую новые экземпляры:

```
constexpr CX create_cx()
{
    return CX();
}
```

Можно также создать простую `constexpr`-функцию, копирующую свой параметр:

```
constexpr CX clone(CX val)
{
    return val;
}
```

Но это почти все, что позволено сделать в стандарте C++11, — `constexpr`-функция может только вызывать другие `constexpr`-функции. В стандарте C++14 это ограничение снято, и в `constexpr`-функции можно делать почти все что угодно, при условии, что она не изменяет никакие объекты с нелокальной областью видимости. Но даже в рамках стандарта C++11 можно применить `constexpr` к компонентным функциям и конструктору `CX`:

```
class CX
{
private:
    int a;
    int b;
public:
    CX() = default;
    constexpr CX(int a_, int b_):
        a(a_), b(b_)
    {}
    constexpr int get_a() const ←❶
    {
        return a;
    }
    constexpr int get_b() ←❷
    {
        return b;
    }
    constexpr int foo()
    {
        return a+b;
    }
};
```

Теперь в C++11 квалификатор `const` в функции `get_a()` ❶ избыточен, поскольку его применение и так подразумевается присутствием ключевого слова `constexpr`, а функция `get_b()` и так квалифицируется как `const`, даже притом, что квалификатор `const` опущен ❷. В стандарте C++14 ситуация изменилась (из-за расширенных возможностей `constexpr`-функций), поэтому `get_b()` больше не является подразумеваемо помечаемой как `const`. И теперь можно создавать более сложные `constexpr`-функции наподобие следующей:

```
constexpr CX make_cx(int a)
{
    return CX(a,1);
}
constexpr CX half_double(CX old)
{
    return CX(old.get_a()/2,old.get_b()*2);
}
constexpr int foo_squared(CX val)
{
    return square(val.foo());
}
int array[foo_squared(half_double(make_cx(10)))]; ← 49 элементов
```

Как бы ни было все это интересно, затраченные усилия не соразмерны с замысловатым способом вычисления каких-то границ массива или составной константы. Основным преимуществом константных выражений и `constexpr`-функций, заданных типом, определяемые пользователем, является то, что литеральный тип, проинициализированный константным выражением, является статически проинициализированным, следовательно, его инициализация свободна от возникновения состояний гонки и проблем с порядком проведения инициализации:

CX `si=half_double(CX(42,19));` ← Статически проинициализирован

Это относится также и к конструкторам. Если конструктор объявлен с ключевым словом `constexpr` и параметры конструктора являются константными выражениями, *инициализация* является *константной* и происходит как составная часть фазы статической инициализации. Это одно из важнейших изменений, внесенных в стандарт C++11, с точки зрения развития конкурентности: за счет разрешения конструкторов, определяемых пользователем, которые тем не менее могут проходить статическую инициализацию, можно избежать возникновения любых состояний гонки на этапе их инициализации, поскольку они гарантированно будут проинициализированы еще до запуска любого другого кода.

Это особенно существенно для таких классов, как `std::mutex` (см. подраздел 3.2.1) или `std::atomic<>` (см. подраздел 5.2.6), где может потребоваться использование глобальных экземпляров для синхронизации доступа к другим переменным и при этом не допустить состояний гонки. Это было бы невозможно, если бы конструктор мьютекса стал предметом состояния гонки, поэтому конструктор по умолчанию `std::mutex` объявлен с ключевым словом `constexpr`, чтобы гарантировать неизменяемую инициализацию мьютекса на этапе статической инициализации.

A.4.2. constexpr-объекты

До сих пор мы говорили только о `constexpr` по отношению к функциям. Но ключевое слово `constexpr` можно также применять и к объектам. Главным образом это делается в целях диагностики. Тем самым проверяется, что объект проинициализирован константным выражением, `constexpr`-конструктором или агрегатным инициализатором, составленным из константных выражений. Кроме того, объект при этом объявляется как `const`:

```
constexpr int i=45;           ← Все правильно
constexpr std::string s("hello"); ← Ошибка; std::string не является
int foo();                   литеральным типом
constexpr int j=foo();       ← Ошибка; foo() не объявлена
                             с ключевым словом constexpr
```

A.4.3. Требования к constexpr-функциям

Чтобы объявить функцию с модификатором `constexpr`, она должна отвечать определенным требованиям. Если она им не отвечает, объявление ее с этим модификато-

ром вызовет ошибку компиляции. В C++11 к constexpr-функциям предъявляются следующие требования:

- ❑ все параметры должны быть литерального типа;
- ❑ возвращаемое значение должно быть литерального типа;
- ❑ тело функции должно состоять из одной инструкции `return`;
- ❑ выражение в инструкции `return` должно быть константным;
- ❑ любой конструктор или оператор преобразования, используемый для построения из выражения возвращаемого значения, должен быть объявлен с модификатором `constexpr`.

Здесь нет ничего необычного. Должна сохраняться возможность встраивания функции в константное выражение, чтобы оно таковым и оставалось, и ничего изменять нельзя. *Constexpr-функции относятся к чистым функциям* и не вызывают побочных эффектов.

В C++14 требования были существенно снижены. Хотя сама идея о чистой функции без побочных эффектов осталась, телу функции разрешено значительно расширить содержимое:

- ❑ разрешено наличие нескольких инструкций `return`;
- ❑ объекты, созданные внутри функции, можно изменять;
- ❑ допускается применение циклов, условных выражений и коммутарующих инструкций.

К constexpr-функциям, являющимся компонентами класса, предъявляются дополнительные требования:

- ❑ constexpr-функции, являющиеся компонентами класса, не могут быть виртуальными;
- ❑ класс, для которого функция является компонентной, должен быть литерального типа.

К constexpr-конструкторам применяются другие правила:

- ❑ для компилятора C++11 тело конструктора должно быть пустым; для компилятора C++14 и компиляторов более поздних версий конструктор должен отвечать требованиям, предъявляемым к constexpr-функциям;
- ❑ каждый базовый класс должен быть проинициализирован;
- ❑ каждые не-`static` компонентные данные должны быть проинициализированы;
- ❑ любые выражения, используемые в списке инициализации компонента, должны квалифицироваться как константные выражения;
- ❑ конструкторы, выбранные для инициализации компонентных данных и базовых классов, должны быть constexpr-конструкторами;
- ❑ любой конструктор или оператор преобразования, используемый для построения компонентных данных и базовых классов из соответствующего выражения инициализации, должен быть объявлен с модификатором `constexpr`.

Это те же правила, что и для функций, за исключением отсутствия возвращаемого значения, поэтому инструкция `return` отсутствует. Вместо этого конструктор инициализирует все базовые компоненты и компоненты данных, имеющиеся в списке инициализации компонентов. Тривиальные копирующие конструкторы подразумеваемо являются `constexpr`-конструкторами.

A.4.4. `constexpr` и шаблоны

Когда модификатор `constexpr` применяется к шаблону функции или к компонентной функции шаблона класса, он игнорируется, если параметры или возвращаемые типы конкретного экземпляра шаблона не являются литеральными типами. Это позволяет создавать шаблоны функций, которые становятся `constexpr`-функциями при соответствующем типе параметров шаблона и обычными встраиваемыми функциями в ином случае, например:

```
template<typename T>
constexpr T sum(T a, T b)
{
    return a+b;
}
constexpr int i=sum(3,42);
std::string s=
    sum(std::string("hello"),
        std::string(" world"));
```

Все верно; `sum<int>` относится к `constexpr`

Все верно, но `sum<std::string>` не относится к `constexpr`

Функция должна удовлетворять всем другим требованиям, предъявляемым к `constexpr`-функциям. Объявлять `constexpr`-функцию с несколькими инструкциями только потому, что она является шаблоном функции, нельзя. Это неизменно вызовет ошибку компиляции.

A.5. Лямбда-функции

Благодаря своим возможностям существенно упростить код и избавить его от шаблонности, связанной с созданием вызываемых объектов, лямбда-функции считаются одним из наиболее интересных функций стандарта C++11. Синтаксис лямбда-функций C++11 позволяет определять функцию там, где она необходима в другом выражении. Он хорошо вписывается в предикаты, предоставляемые функциям ожидания `std::condition_variable` (как в примере из подраздела 4.1.1), поскольку позволяет быстро выражать семантику в понятиях доступных переменных, вместо того чтобы перехватывать необходимое состояние в компонентных переменных класса с помощью оператора вызова функции.

В простейшем случае *лямбда-выражение* определяет автономную функцию, не получающую параметров и полагающуюся только на глобальные переменные и функции. Ей даже не нужно возвращать значение. Такое лямбда-выражение представляет собой ряд инструкций, заключенных в фигурные скобки, перед которыми стоят квадратные скобки (*лямбда-интродуктор*):

```

[] {
    do_stuff();
    do_more_stuff();
}();

```

← Лямбда-выражение начинается с []

← Завершение лямбда-выражения и его вызов

В этом примере лямбда-выражение тут же вызывается за счет того, что сразу за ним следуют круглые скобки, но обычно так не делается. Ведь для непосредственного вызова можно обойтись и без лямбда-выражения и внести инструкции непосредственно в исходный код. Чаще всего лямбда-выражение передается в качестве параметра шаблону функции, получающему в качестве одного из своих параметров вызываемый объект. В таком случае лямбда-выражению, вероятно, нужно будет получить параметры, или иметь возвращаемое значение, или использовать и то и другое. Если нужно получить параметры, их список следует указать сразу же за лямбда-интродуктором, как это делается с обычной функцией. Например, следующий код записывает все элементы вектора `std::cout`, используя в качестве разделителей символы новой строки:

```

std::vector<int> data=make_data();
std::for_each(data.begin(),data.end(),[](int i){std::cout<<i<<"\n";});

```

Практически так же легко решить вопрос с возвращаемым значением. Если тело лямбда-функции состоит из одной инструкции `return`, возвращаемым типом лямбда-функции является тип возвращаемого выражения. Например, простой лямбда-функцией, показанной в листинге А.4, можно воспользоваться для ожидания установки флага с помощью `std::condition_variable` (см. подраздел 4.1.1).

Листинг А.4. Простая лямбда-функция с автоматически выводимым возвращаемым типом

```

std::condition_variable cond;
bool data_ready;
std::mutex m;
void wait_for_data()
{
    std::unique_lock<std::mutex> lk(m);
    cond.wait(lk,[] {return data_ready;}); ←❶
}

```

Возвращаемый тип лямбда-функции, переданный `cond.wait()` ❶, выводится из типа `data_ready` и в результате является булевым. Когда условная переменная пробуждается от ожидания, она вызывает лямбда-функцию с заблокированным мьютексом и возвращается только после вызова `wait()`, когда `data_ready` имеет значение `true`.

А если создать тело лямбда-функции из одной инструкции `return` не представляется возможным? В таком случае придется указать тип возвращаемого значения явным образом. Это можно сделать, даже если тело функции состоит из одной инструкции `return`, но вы *должны* сделать это, если тело лямбда-функции представляет собой более сложную конструкцию. Тип возвращаемого значения указывается после списка параметров лямбда-функции за стрелкой (`->`). Если лямбда-функция не получает никаких параметров, для явного указания типа возвращаемого значения

нужно все равно выключить пустой список параметров. Наш предикат условной переменной можно записать следующим образом:

```
cond.wait(lk,[]()->bool{return data_ready;});
```

Указав возвращаемый тип, можно расширить область применения лямбда-функции до работы с регистрационными сообщениями или до выполнения более сложной обработки данных:

```
cond.wait(lk,[]()->bool{
    if(data_ready)
    {
        std::cout<<"Data ready"<<std::endl;
        return true;
    }
    else
    {
        std::cout<<"Data not ready, resuming wait"<<std::endl;
        return false;
    }
});
```

Показанные здесь лямбда-функции весьма эффективны и могут существенно упростить код, но наибольшая эффективность таких функций проявляется при захвате локальных переменных.

A.5.1. Лямбда-функции, ссылающиеся на локальные переменные

Лямбда-функции с *интродуктором* `[]` не могут ссылаться на локальные переменные из области видимости. Им разрешено использовать только глобальные переменные и все, что передано в качестве параметра. Если нужен доступ к локальной переменной, ее следует захватить. Проще всего это сделать путем *захвата* всего набора переменных внутри локальной области видимости, используя лямбда-интродуктор, имеющий вид `[=]`. Вот, собственно, и все, что нужно сделать, — теперь ваша лямбда-функция может обращаться к *копиям* локальных переменных, сделанных на момент создания лямбда-функции.

Чтобы посмотреть на это в действии, рассмотрим следующий пример простой функции:

```
std::function<int(int)> make_offseter(int offset)
{
    return [=](int j){return offset+j;};
}
```

Каждый вызов `make_offseter` возвращает посредством функции-оболочки `std::function<>` новый объект лямбда-функции. Эта возвращаемая функция добавляет предоставленное смещение к любому переданному ей параметру. Например, при выполнении кода:


```
int main()
{
    std::function<int(int)> offset_42=make_offsetter(42);
    std::function<int(int)> offset_123=make_offsetter(123);
    std::cout<<offset_42(12)<<" "<<offset_123(12)<<std::endl;
    std::cout<<offset_42(12)<<" "<<offset_123(12)<<std::endl;
}
```

Числа 54, 135 будут выведены дважды, поскольку функция, возвращаемая из первого вызова `make_offsetter`, всегда прибавляет 42 к предоставленному аргументу, а функция, возвращаемая из второго вызова `make_offsetter`, всегда прибавляет к предоставленному аргументу 123.

Это наиболее безопасная форма захвата локальной переменной; здесь все копируется, что позволяет вернуть лямбда-функцию и вызвать ее за пределами области видимости функции по умолчанию. Но это не единственный вариант захвата; вместо этого можно все захватить по ссылке. В таком случае, если переменные, на которые ссылается лямбда-функция, были уничтожены из-за выхода из области видимости функции или блока, которым они принадлежат, вызов лямбда-функции может привести к неопределенному поведению, точно так же, как приводит к неопределенному поведению ссылка на уже уничтоженную переменную при любых других обстоятельствах.

Как показано в следующем примере, лямбда-функция, захватывающая все локальные переменные по ссылке, обозначается при создании с помощью интродуктора `[&]`:

```
int main()
{
    int offset=42; ①
    std::function<int(int)> offset_a=[&](int j){return offset+j;}; ②
    offset=123; ③
    std::function<int(int)> offset_b=[&](int j){return offset+j;}; ④
    std::cout<<offset_a(12)<<" "<<offset_b(12)<<std::endl; ⑤
    offset=99; ⑥
    std::cout<<offset_a(12)<<" "<<offset_b(12)<<std::endl; ⑦
}
```

В функции `make_offsetter` из предыдущего примера для захвата копии смещения использовался лямбда-интродуктор `[=]`, а в функции `offset_a` в этом примере для захвата смещения по ссылке используется лямбда-интродуктор `[&]` ②. Неважно, что значением смещения по умолчанию является 42 ①; результат вызова `offset_a(12)` всегда будет зависеть от текущего значения смещения `offset`. Перед созданием второй (идентичной) лямбда-функции `offset_b` ④ значение `offset` изменяется на 123 ③, и эта вторая лямбда-функция опять захватывает смещение по ссылке, следовательно, результат зависит от текущего значения `offset`.

Теперь в первой строке вывода ⑤ `offset` все еще имеет значение 123, поэтому вывод имеет вид 135, 135. Но во второй строке вывода ⑦ `offset` был изменен на 99 ⑥, поэтому теперь вывод имеет вид 111, 111. Обе функции, `offset_a` и `offset_b`, прибавляют текущее значение `offset` (99) к предоставленному аргументу (12).

По благодаря особенностям C++ выбрать между всем или ничем не приходится. Можно выбрать вариант, при котором некоторые переменные будут захватываться по копии, а некоторые — по ссылке, и можно путем настройки лямбда-интродуктора захватывать только явно выбранные переменные. Если нужно *скопировать* все используемые переменные, за исключением одной или двух, можно воспользоваться лямбда-интродуктором вида [=], но за знаком равенства поставить список переменных, захватываемых по ссылке, перед которыми поставить знаки амперсанда. Следующий пример выведет число 1239, потому что *i* копируется в лямбда-функцию, а *j* и *k* захватываются по ссылке:

```
int main()
{
    int i=1234, j=5678, k=9;
    std::function<int()> f=[=, &j, &k]{return i+j+k;};
    i=1;
    j=2;
    k=3;
    std::cout<<f()<<std::endl;
}
```

Можно сделать и по-другому: захватить по ссылке переменные по умолчанию, а указанный поднабор переменных захватить копированием. В таком случае используется лямбда-интродуктор вида [&], но за амперсандом следует список переменных, захватываемых копированием. Следующий пример выведет число 5688, потому что *i* захватывается по ссылке, а *j* и *k* копируются:

```
int main()
{
    int i=1234, j=5678, k=9;
    std::function<int()> f=[&, j, k]{return i+j+k;};
    i=1;
    j=2;
    k=3;
    std::cout<<f()<<std::endl;
}
```

Если нужно захватить только именованные переменные, то можно указать лишь список захватываемых переменных, а ведущий символ = или & опустить. При этом, если нужна не копия, а захват по ссылке, перед именами соответствующих переменных следует поставить знак амперсанда. Следующий код выведет число 5682, потому что переменные *i* и *k* захвачены по ссылке, а переменная *j* скопирована:

```
int main()
{
    int i=1234, j=5678, k=9;
    std::function<int()> f=[&i, j, &k]{return i+j+k;};
    i=1;
    j=2;
    k=3;
    std::cout<<f()<<std::endl;
}
```


Последний вариант позволяет обеспечить захват только нужных переменных, поскольку любая ссылка на локальную переменную, отсутствующую в списке захвата, повлечет за собой ошибку компиляции. Выбор этого варианта заставляет проявлять осторожность при доступе к компонентам класса, если функция, содержащая лямбда-функцию, является компонентной. Компоненты класса не могут захватываться напрямую. Если доступ к ним нужен из внешней лямбда-функции, придется захватывать указатель `this`, добавляя его в список захвата. В следующем примере лямбда-функция захватывает `this`, открывая тем самым доступ к компоненту класса `some_data`:

```
struct X
{
    int some_data;
    void foo(std::vector<int>& vec)
    {
        std::for_each(vec.begin(), vec.end(),
            [this](int& i){i+=some_data;});
    }
};
```

В контексте конкурентности наибольшая польза от применения лямбда-функций проявляется в их использовании в качестве предиката для `std::condition_variable::wait()` (см. подраздел 4.1.1) и с `std::packaged_task<>` (см. подраздел 4.2.1) или с пулами потоков для упаковки небольших задач. Их также можно передать в качестве функции потока конструктору `std::thread` (см. подраздел 2.1.1) и в качестве просто функции при использовании таких параллельных алгоритмов, как `parallel_for_each()` (из подраздела 8.5.1).

С появлением стандарта C++14 лямбда-функции могут также быть *обобщенными*, где типы параметров объявляются как `auto`, а не как указанный тип. В таком случае оператор вызова функции подразумевается является шаблоном и тип параметра при вызове лямбда-функции выводится из представленного аргумента. Например:

```
auto f=[](auto x){ std::cout<<"x="<<x<<std::endl;};
f(42);           // x относится к типу int; выводится "x=42"
f("hello");     // x относится к типу const char*; выводится "x=hello"
```

В стандарт C++14 также добавлено понятие *обобщенных захватов*, позволяющее захватывать результаты выражений, а не непосредственную копию локальной переменной или ссылку на нее. Чаще всего это может пригодиться при захвате типов, предназначенных только для перемещения, путем их перемещения, а не при обязательном захвате их по ссылке. Например:

```
std::future<int> spawn_async_task(){
    std::promise<int> p;
    auto f=p.get_future();
    std::thread t([p=std::move(p)](){ p.set_value(find_the_answer());});
    t.detach();
    return f;
}
```

Здесь промисс переменен в лямбда-функцию путем обобщенного захвата `p=std::move(p)`, следовательно, можно безопасно отключиться от потока, не тревожась о появлении висячей ссылки на уничтоженную локальную переменную. Теперь, после создания лямбда-функции, исходная переменная `p` находится в состоянии «только для перемещения», что вынуждает получить фьючерс заранее.

А.6. Вариативные шаблоны

Вариативными считаются шаблоны с переменным числом параметров. Так же как ранее была возможность располагать вариативными функциями, например `printf`, получающими переменное число параметров, теперь можно располагать и вариативными шаблонами, имеющими переменное число параметров *шаблона*. Вариативные шаблоны повсеместно используются в C++ Thread Library. Например, вариативным функциональным шаблоном является конструктор для запуска потока `std::thread` (см. подраздел 2.1.1), к таким шаблонам также относится и `std::packaged_task<>` (см. подраздел 4.2.2). С пользовательской точки зрения достаточно знать, что шаблон принимает неограниченное число параметров, но, если захочется создать такой шаблон или же появится интерес к тому, как это все работает, без подробностей не обойтись.

Точно так же, как вариативные функции объявляются с помощью многоточия (...) в списке параметров функции, вариативные шаблоны объявляются с помощью многоточия в списке параметров шаблона:

```
template<typename ... ParameterPack>
class my_template
{};
```

Вариативные шаблоны можно также использовать для частичной специализации шаблона, даже если первичный шаблон не был вариативным. Например, первичный шаблон для `std::packaged_task<>` (см. подраздел 4.2.1) относится к простым шаблонам с одним параметром:

```
template<typename FunctionType>
class packaged_task;
```

Но этот первичный шаблон нигде не определяется; это всего лишь прототип для частичной специализации:

```
template<typename ReturnType, typename ... Args>
class packaged_task<ReturnType(Args...)>;
```

В этой частичной специализации и содержится реальное определение класса. В главе 4 было показано, что для объявления задачи, получающей при ее вызове в качестве параметров объекты типов `std::string` и `double` и предоставляющей результат посредством объекта `std::future<int>`, можно воспользоваться кодом `std::packaged_task<int(std::string,double)>`.

В данном объявлении демонстрируются еще две особенности вариативных шаблонов. Первая из них относительно проста: наряду с вариативными (*Args*) в том же объявлении можно указывать и обычные параметры (например, *ReturnType*). Вторая особенность продемонстрирована в использовании в качестве списка аргументов специализации шаблона *Args...*, чтобы показать, что типы, составляющие *Args* при создании экземпляра шаблона, будут перечислены в этом месте. Поскольку это частичная специализация, данный код работает как сопоставление с образцом; типы, появляющиеся в реальном контексте в экземпляре, захватываются как *Args*. Вариативный параметр *Args* называется *пакетом параметров*, а использование *Args...* — *расширением пакета*.

Как и в вариативных функциях, вариативная часть может быть пустой или же может иметь множество записей. Например, в `std::packaged_task<my_class()>` параметром *ReturnType* является *my_class*, а пакет параметров *Args* пуст, в `std::packaged_task<void(int,double,my_class&,std::string*)>` параметром *ReturnType* является *void*, а пакет параметров *Args* — списком из *int*, *double*, *my_class&*, *std::string**.

А.6.1. Расширение пакета параметров

Эффективность вариативных шаблонов проявляется в том, что можно сделать с расширением пакета параметров: здесь нет ограничений по расширению списка типов в его исходный вид. В первую очередь, расширением пакета параметров можно воспользоваться напрямую везде, где требуется список типов, например в списке аргументов для другого шаблона:

```
template<typename ... Params>
struct dummy
{
    std::tuple<Params...> data;
};
```

В данном случае единственный вариативный компонент *data* является экземпляром `std::tuple<>`, содержащим все указанные типы, следовательно, у `dummy<int,double,char>` имеется компонентный тип `std::tuple<int,double,char>`. Расширения пакетов параметров можно использовать с обычными типами:

```
template<typename ... Params>
struct dummy2
{
    std::tuple<std::string,Params...> data;
};
```

На этот раз у класса *tuple* есть дополнительный (первый) компонент типа `std::string`. Здесь привлекает возможность создания схемы с расширением пакета параметров, которая затем копируется для каждого имеющегося в расширении элемента. Для этого многоточие (...), обозначающее расширение пакета параметров,

помещается в конец схемы. Например, вместо простого создания кортежа элементов, предоставленных в пакете параметров, можно создать кортеж указателей на элементы или даже кортеж из интеллектуальных указателей `std::unique_ptr<>` на ваши элементы:

```
template<typename ... Params>
struct dummy3
{
    std::tuple<Params* ...> pointers;
    std::tuple<std::unique_ptr<Params> ...> unique_pointers;
};
```

Выражение, в котором дается описание типа, может быть сколь угодно сложным, при условии, что пакет параметров входит в состав этого выражения и что за выражением следует многоточие, обозначающее расширение. При расширении пакета параметров для каждой записи в пакете имеющийся в ней тип подставляется в выражение описания типа для создания соответствующего элемента в получающемся списке. Если ваш пакет параметров `Params` содержит типы `int`, `int`, `char`, то в результате расширения, указанного в `std::tuple<std::pair<std::unique_ptr<Params>, double> ...>`, получится `std::tuple<std::pair<std::unique_ptr<int>, double>, std::pair<std::unique_ptr<int>, double>, std::pair<std::unique_ptr<char>, double>>`. Если расширение пакета используется в качестве списка аргументов шаблона, то в этом шаблоне не должны быть вариативные параметры, но если это условие соблюдено, то размер пакета должен в точности совпадать с числом требуемых параметров шаблона:

```
template<typename ... Types>
struct dummy4
{
    std::pair<Types...> data;
};
dummy4<int, char> a;
dummy4<int> b;
dummy4<int, int, int> c;
```

Все верно, data относится к типу std::pair<int, char>
 Ошибка; отсутствует второй тип
 Ошибка; слишком много типов

Второй способ применения расширения пакета параметров подойдет для объявления списка параметров функции:

```
template<typename ... Args>
void foo(Args ... args);
```

Здесь создается новый пакет параметров `args`, являющийся списком параметров функции, а не списком типов, и его, как и прежде, можно расширить, указав многоточие. Теперь схемой с расширением пакета можно воспользоваться для объявления параметров функции так же, как и при расширении пакета в любом другом месте. Например, такая схема используется конструктором `std::thread` для получения всех аргументов функции по ссылке на n -значение (см. раздел А.1):

```
template<typename CallableType, typename ... Args>
thread::thread(CallableType&& func, Args&& ... args);
```

Затем пакет параметров функции может использоваться для вызова другой функции путем указания расширения пакета в списке аргументов вызываемой функции.

Как и в случае с расширениями типов, схемой можно воспользоваться для каждого выражения в списке получающихся аргументов. Например, одна широко распространенная идиома применения ссылок на *r*-значения заключается в использовании `std::forward<>` для предотвращения присутствия в предоставленных аргументах функции всего, что не является *r*-значениями:

```
template<typename ... ArgTypes>
void bar(ArgTypes&& ... args)
{
    foo(std::forward<ArgTypes>(args)...);
}
```

Следует отметить, что в данном случае расширение пакета содержит как пакет типов `ArgTypes`, так и пакет параметров функции `args`, а многоточие указано после всего выражения. Если вызвать такой вот `bar`:

```
int i;
bar(i, 3.141, std::string("hello"));
```

то результат расширения примет следующий вид:

```
template<
void bar<int&, double, std::string>(
    int& args_1,
    double&& args_2,
    std::string&& args_3)
{
    foo(std::forward<int&>(args_1),
        std::forward<double>(args_2),
        std::forward<std::string>(args_3));
}
```

Здесь первый аргумент вполне корректно передается `foo` в качестве ссылки на *r*-значение, а другие аргументы передаются в качестве ссылок на *r*-значения.

И последнее, что можно сделать с расширением пакета параметров, — это определить его длину с помощью оператора `sizeof...`. Все очень просто: `sizeof...(p)` возвращает число элементов в пакете параметров `p`. И неважно, является ли этот пакет пакетом параметров или пакетом аргументов функции; результат будет одинаковым. И это, наверное, единственный случай, когда можно будет воспользоваться пакетом параметров и не поставить за ним многоточие, поскольку оно уже является частью оператора `sizeof...`. Следующая функция возвращает число предоставленных ей аргументов:

```
template<typename ... Args>
unsigned count_args(Args ... args)
{
    return sizeof... (Args);
}
```

Как и в случае применения обычного оператора `sizeof`, результатом выполнения `sizeof...` является константное выражение, следовательно, им можно воспользоваться для задания границ массива и т. д.

A.7. Автоматическое выведение типа переменной

C++ является статически типизированным языком: на время компиляции тип каждой переменной уже известен. Более того, программист обязан указать тип каждой переменной. В некоторых случаях из-за этого могут получаться весьма громоздкие имена. Например:

```
std::map<std::string, std::unique_ptr<some_data>>> m;
std::map<std::string, std::unique_ptr<some_data>>::iterator
    iter=m.find("my key");
```

Обычно в качестве решения, позволяющего сократить длину идентификаторов типов и по возможности снизить количество проблем, возникающих из-за несовместимости типов, использовались псевдонимы типов (**typedef**). Этот прием работает и в C++11, но в данном стандарте появился и новый прием: если переменная инициализируется в своем определении значением того же типа, то в качестве типа можно указать **auto**. В таком случае компилятор автоматически выведет тип переменной, который станет таким же, как и у инициализировавшего ее значения. Пример итератора можно записать в виде:

```
auto iter=m.find("my key");
```

Теперь обычное ключевое слово **auto** можно украсить объявлением **const**-переменных, или указателя, или ссылочных переменных. Рассмотрим несколько примеров объявления переменных, использующих **auto** и соответствующий тип переменной:

```
auto i=42;           // int
auto& j=i;           // int&
auto const k=i;      // int const
auto* const p=&i;     // int * const
```

Правила выведения типа переменной основаны на правилах, применяемых для еще одного места в языке, где выводятся типы: в параметрах шаблонов функций. В объявлении вида:

```
some-type-expression-involving-auto var=some-expression;
```

тип **var** будет таким же, как и тип, выведенный для параметра шаблона функции с тем же выражением типа, за исключением замены **auto** именем параметра типа шаблона:

```
template<typename T>
void f(type-expression var);
f(some-expression);
```

Это означает, что типы массивов сводятся к указателям, а ссылки опускаются, если только в выражении типа переменная не объявлена явным образом как ссылка. Например:

```
int some_array[45];
auto p=some_array;    // int*
int& r=*p;
auto x=r;             // int
auto& y=r;            // int&
```

Это может существенно упростить объявление переменных, в частности там, где полноценный идентификатор типа слишком длинный или, возможно, даже неизвестен (например, тип результата вызова функции в шаблоне).

A.8. Локальные переменные потока

Локальные переменные потока позволяют располагать отдельными экземплярами переменной для каждого потока в вашей программе. Переменная помечается как локальная переменная потока путем ее объявления с ключевым словом `thread_local`. Локальными по отношению к потоку могут объявляться переменные в области видимости пространства имен, статические компонентные данные класса и локальные переменные, и о них говорят, что имеют *потокосвое время жизни* (thread storage duration):

```
thread_local int x;
class X
{
    static thread_local std::string s;
};
static thread_local std::string X::s;
void foo()
{
    thread_local std::vector<int> v;
}
```

Локальные переменные потока в области видимости пространства имен

Статические компонентные данные класса, являющиеся локальными по отношению к потоку

Требуется определение X::s

Локальная переменная потока

Локальные переменные потока в области видимости пространства имен и локальные по отношению к потоку статические компонентные данные класса создаются перед первым использованием локальной переменной потока из той же единицы компиляции, но *когда именно* — не оговаривается. В некоторых реализациях локальные переменные потока можно создавать при запуске потока; а в некоторых — сразу же перед их первым использованием в каждом потоке, бывает и так, что их создают и в другие определенные моменты времени или же при каких-то обстоятельствах в зависимости от контекста их использования. Конечно же, если из конкретной единицы компиляции никакие локальные переменные потока не используются, нет никаких гарантий, что они вообще будут созданы. Это позволяет производить динамическую загрузку модулей, содержащих локальные переменные потока. Такие переменные можно создавать для конкретного потока при первой же его ссылке на локальную переменную потока из динамически загруженного модуля.

Локальные переменные потока, объявленные внутри функции, инициализируются при первом же проходе потока управления через их объявление в конкретном потоке. Если функция конкретным потоком не объявляется, то все локальные переменные потока, объявляемые в этой функции, не создаются. Это точно такое же поведение, которое проявляется в отношении локальных статических переменных, за исключением того, что оно применяется отдельно к каждому потоку.

У локальных переменных потока есть и другие свойства, присущие статическим переменным: перед любой предстоящей инициализацией (например, динамической) они проходят нулевую инициализацию и если при создании локальной переменной потока выдается исключение, то для прекращения выполнения приложения вызывается функция `std::terminate()`.

Деструкторы для всех локальных переменных потока, созданных в конкретном потоке, запускаются при возвращении из функции потока в порядке, обратном созданию этих переменных. Поскольку порядок инициализации не определяется, то важно обеспечить отсутствие взаимозависимостей между деструкторами таких переменных. Если выход из деструктора локальной переменной потока выполняется с выдачей исключения, то, как и для конструктора, вызывается функция `std::terminate()`.

Локальные переменные потока также уничтожаются для потока, если этот поток вызывает функцию `std::exit()` или возвращается из функции `main()` (что эквивалентно вызову `std::exit()` с возвращаемым значением функции `main()`). Если при выходе из приложения продолжают выполняться другие потоки, деструкторы локальных переменных потока в таких потоках не вызываются.

Хотя у каждого потока адреса для локальных переменных разные, возможность получить обычный указатель на эти переменные все же есть. Затем указатель ссылается на объект в том потоке, который получает адрес, и может использоваться, позволяя другим потокам обращаться к этому объекту. Обращение к объекту после его уничтожения вызывает (как и всегда) неопределенное поведение, поэтому, если указатель на локальную переменную потока передается другому потоку, нужно проследить за тем, чтобы он не разыменовывался после завершения того потока, которому он принадлежит.

А.9. Выведение аргументов шаблона класса

Стандарт C++17 расширяет замысел автоматического вывода типов параметров шаблона: если объявляется объект шаблонного типа, то во многих случаях тип параметров шаблона можно вывести из инициализатора объекта. В частности, если объект объявляется с именем шаблона класса, без указания списка аргументов шаблона, то конструкторы, объявленные в шаблоне класса, используются для вывода аргументов шаблона из инициализатора объекта в соответствии с обычными правилами вывода типов для шаблонов функций.

Например, `std::lock_guard` получает единственный параметр шаблона, который является типом мьютекса. Конструктор также получает единственный параметр, ссылающийся на этот тип. Если объявить, что объект относится к типу `std::lock_guard`, то тип параметра можно вывести из типа предоставленного мьютекса:

```
std::mutex m;
std::lock_guard guard(m); // выводится std::lock_guard<std::mutex>
```

Этот же принцип применим и к шаблону класса `std::scoped_lock`, за исключением того, что у него есть несколько параметров шаблона, которые можно вывести из нескольких мьютексных аргументов:

```
std::mutex m1;
std::shared_mutex m2;
std::scoped_lock guard(m1,m2);
// выводится std::scoped_lock<std::mutex,std::shared_mutex>
```

Для тех шаблонов, у которых конструкторы неправильно выводят типы, автор шаблона может написать явные руководства по выводу, гарантируя правильный вывод типов. Но такие возможности выходят за рамки данной книги.

Резюме

В этом приложении возможности языка, введенные стандартом C++11, рассмотрены поверхностно, поскольку мы разобрали только свойства, активно используемые в Thread Library. К другим новым свойствам языка, наряду с многочисленными незначительными изменениями, можно отнести статические утверждения, строго типизированные перечисления, делегирование конструкторов, поддержку Unicode, псевдонимы шаблонов и новую универсальную последовательность инициализации. Подробное описание всех новых свойств не входило в задачи написания данной книги и заслуживает отдельного издания. Существенное количество изменений было привнесено в стандарты C++14 и C++17, но опять же они выходят за рамки данной книги. На мой взгляд, наилучший обзор всего набора изменений, внесенных в стандарт на момент написания этой книги, дан в документации на сайте <http://www.cppreference.com>, а также в ответах на наиболее часто задаваемые вопросы (FAQ) по C++11 Бьерна Страуструпа (Bjarne Stroustrup) (<http://www.research.att.com/~bs/C++0xFAQ.html>), хотя можно также в скором времени рассчитывать на соответствующую актуализацию популярных справочников по C++.

Надеюсь, что краткого изложения новых свойств в этом приложении было вполне достаточно, чтобы показать, какое отношение они имеют к Thread Library, позволить вам создавать многопоточный код и разбираться в чужих кодах, использующих новые свойства. Хотя приложение дает достаточное представление для простого использования рассмотренных свойств, это лишь краткое введение, а не полный справочник или руководство по их использованию. Если есть намерение интенсивно применять эти новые свойства, я рекомендую приобрести соответствующий справочник или руководство.